Python Engineering

Class 07 - NumPy Arrays | EX15b - Shapes & Operations Practice

No cambiamos de week ni de class. Seguimos en **week_02_data_structures -> class_07_numpy_arrays**. Este ejercicio extra busca ganar soltura con arrays antes de avanzar.

Objetivo

Practicar deliberadamente como se crean arrays 1D y 2D, como se inspeccionan sus dimensiones, como se indexan, que operaciones son compatibles y que significa que dos shapes concuerden.

BLOCK A - Crear arrays e inspeccionar su geometria

```
import numpy as np

vector_a = np.array([1.0, 2.0, 3.0])
vector_b = np.array([4.0, 5.0, 6.0])

matrix_a = np.array([
    [1.0, 2.0, 3.0],
    [4.0, 5.0, 6.0]
])

matrix_b = np.array([
    [10.0, 20.0, 30.0],
    [40.0, 50.0, 60.0]
])
```

Para cada objeto imprime: `shape`, `ndim`, `size` y `dtype`.

Antes de ejecutar, intenta predecir los resultados. Esa prediccion es parte del ejercicio.

BLOCK B - Indexacion y slicing

```
matrix_a[0, 0]
matrix_a[1, 2]
matrix_a[0, :]
matrix_a[:, 1]
matrix_a[-1, :]
matrix_a[:, 1:]
```

Para cada expresion escribe primero que crees que devuelve y que shape tendra el resultado.

BLOCK C - Operaciones elemento a elemento

```
vector_a + vector_b
vector_a - vector_b
vector_a * vector_b
vector_a / vector_b
vector_a**2
matrix_a + matrix_b
matrix_a * matrix_b
matrix_a * 10
```

Aqui * significa multiplicacion elemento a elemento. Todavia no es producto matricial.

BLOCK D - Broadcasting

```
offset = np.array([100.0, 200.0, 300.0])

result = matrix_a + offset
```

Predice el resultado. NumPy puede aplicar el vector de shape `(3,)` a cada fila de una matriz de shape `(2,3)`. Eso es **broadcasting**.

```
bad_offset = np.array([10.0, 20.0])
```

```
# Try deliberately:
matrix_a + bad_offset
```

Quiero que observes el error y expliques por que los shapes `(2,3)` y `(2,)` no son compatibles en este caso.

BLOCK E - Reshape y transpose

```
row = vector_a.reshape(1, 3)
column = vector_a.reshape(3, 1)

matrix_t = matrix_a.T
```

Comprueba los shapes de `vector_a`, `row`, `column` y `matrix_t`.

Objeto	Shape esperado
vector_a	(3,)
row	(1,3)
column	(3,1)
matrix_a	(2,3)
matrix_a.T	(3,2)

BLOCK F - Concatenacion

```
new_row = np.array([[7.0, 8.0, 9.0]])

matrix_rows = np.vstack((matrix_a, new_row))

new_column = np.array([[10.0], [20.0]])

matrix_columns = np.hstack((matrix_a, new_column))
```

Comprueba como cambia el shape. En `vstack` deben coincidir columnas; en `hstack` deben coincidir filas.

BLOCK G - Maximos por eje

```
np.max(matrix_b)
np.max(matrix_b, axis=0)
np.max(matrix_b, axis=1)

np.argmax(matrix_b)
np.argmax(matrix_b, axis=0)
np.argmax(matrix_b, axis=1)
```

No memorices los resultados: interpreta que dimension estamos reduciendo en cada axis.

BONUS - Producto escalar y producto matriz-vector

```
dot_product = vector_a @ vector_b

matrix_vector = matrix_a @ vector_b
```

El operador @ representa multiplicacion lineal/matricial, no multiplicacion elemento a elemento. Trataremos algebra lineal con mas profundidad mas adelante.

VALIDACION FINAL

```
Check and explain:
- Why (3,) is not the same as (1,3)
- Why (2,3) + (3,) works
- Why (2,3) + (2,) fails
- Why vstack needs the same number of columns
- Why hstack needs the same number of rows
- Difference between * and @
- Difference between axis=0 and axis=1
```

Criterio para cerrar EX15b

No busco que memorices NumPy. El ejercicio estara completado cuando puedas mirar dos shapes y razonar si una operacion tiene sentido antes de ejecutarla.